

Git & GitHub Cheat Sheet

Shrnutí 3denního kurzu – kdy co použít, a přesné příkazy

Na začátku dne (než začneš pracovat)

git status – zjistí, jestli nemáš z minulá rozpracovaná/necommitnutá změna. Pozor: pokud ano, vyřeš je (commit/stash) dřív, než uděláš pull nebo přepneš větve.	git status
git fetch origin – stáhne novinky z GitHubu, ale nic lokálně nemění. Bezpečný first look – můžeš si prohlédnout, co přibýlo, než to sloučíš.	git fetch origin
git pull – rovnou stáhne a sloučí novinky do tvé aktuální větve. Pozor: s necommitnutými změnami může selhat nebo vytvořit konflikt.	git pull
git branch – over si, na které větvi opravdu jsi (hvězdička). Pozor: snadno se stane, že začneš pracovat omylem na main místo na feature.	git branch
git rebase main – pokračuješ-li ve starší feature větvi, srovnej ji s aktuálním main. Menší, včasnější konflikty než jeden velký na konci. Pozor: jen na vlastní, ještě nikam nesdílené větvi.	git checkout feature/xyz git rebase main

Obecný pracovní postup

- Aktualizuj main (fetch/pull), zkontroluj git status
- Vytvoř / přepni se na feature větev pro daný úkol
- Pracuj po malých krocích – commituj často, 1 commit = 1 srozumitelná změna
- Před otevřením PR: rebase na aktuální main (čistá historie)
- Pushni větev, otevři Pull Request na GitHubu
- Po review slouč/squashni do main, smaž feature větev
- Lokálně: checkout main → pull → pokračuj dalším úkolem

Přidávání více změněných souborů

Konkrétní soubory jmenovitě
git add soubor1.py soubor2.py
Vše v aktuálním adresáři a podsložkách
git add .
Úplně všechny změny v repu (i smazané soubory, i mimo aktuální složku)
git add -A
Kontrola PŘED commitem, co je stagované
git status
git diff --staged
Jeden commit pak zahrne všechny stagované soubory najednou
git commit -m "zprava"
Pokročilý: vybrat jen část změn v souboru interaktivně
git add -p

Práce s GitHubem (remote)

Propojit lokální repo s repozitářem na GitHubu	git remote add origin <url>
Přejmenovat výchozí větev na main	git branch -M main
První push nové větve (nastaví sledování)	git push -u origin main
Poslat nové commity	git push
Stáhnout a sloučit novinky z GitHubu	git pull
Jen stáhnout, bez sloučení (bezpečná kontrola)	git fetch origin

Fetch vs. Pull - kam se to vlastně stáhne?

git fetch origin Néstáhne nic do tvé lokální větve ani do souborů na disku. Uloží stažený stav stranou jako tzv. remote-tracking větve, s prefixem origin/ – např. origin/main, origin/feature/xyz. Tvoje lokální main zůstává beze změny.
Co je nového, zjistíš přes
git log origin/main --oneline
git log main..origin/main

Na konci dne (než přestaneš)

Zkontroluj rozpracované změny	git status
Ulož rozpracovanou práci jako commit (i nehotovou, jde upravit později)	git add -A git commit -m "WIP: ..."
Nebo, pokud nechceš commitovat, odlož změny	git stash push -m "popis"
Pošli commity na GitHub (záloha + pokračování odjinud)	git push
Novou větev pushni taky (poprvé s -u)	git push -u origin <nazev-vetve>

Best practices

- Malé, atomické commity – 1 logická změna = 1 commit
- Popisná commit message (co a proč), ne "update"/"fix"
- main/master drž vždy funkční – práci dělej na feature větvích
- Pully/fetchuj často – menší a řídnější konflikty
- git status před i po každé rizikové operaci (reset, rebase, merge)
- Nepřepisuj historii (reset/rebase), kterou už mají stažení jiní

Základy repozitáře

Založit nový lokální repozitář ve složce	git init
Stáhnout existující repozitář z GitHubu	git clone <url>
Co se změnilo / co je stagované	git status
Přidat změny do staging area	git add soubor.py git add -A
Vytvořit commit se změnami ze staging area	git commit -m "zpráva"
Zobrazit historii commitů	git log --oneline git log --oneline --graph --all
Zobrazit nestagovaný / stagovaný rozdíl	git diff git diff --staged

Ignorování souborů

.gitignore v kořeni repa – řádek na vzor, který se nemá sledovat	__pycache__/ *.pyc .vscode/
Soubor už omylem sledovaný? Přestat ho sledovat (zůstane na disku)	git rm --cached soubor

--online
Teprve sloučení posune tvou lokální větev
git checkout main
git merge origin/main
git pull = přesně tohle (fetch + merge) v jednom kroku – proto může kolidovat s necommitnutými změnami, na rozdíl od samotného fetch.
Analogie: fetch = vyzvedneš poštu ze schránky a necháš ji na stole (vidíš ji, ale ještě nezpracovanou). pull = vyzvedneš poštu a rovnou ji založíš do šanonu.

Větve, Pull Requesty a pokročilé nástroje

Větve a merge

Vytvořit a přepnout se na novou větev	git checkout -b feature/xyz
Přepnout se na existující větev	git checkout main
Vypsat lokální větve (* = aktuální)	git branch
Sloučit větev do aktuální (jsi na main → mergeš feature)	git checkout main git merge feature/xyz
Konflikt: uprav soubor (odstraň značky <</===/>>>), pak	git add soubor git commit
Smazat větev po dokončení (lokálně / na GitHubu)	git branch -d feature/xyz git push origin --delete feature/xyz

Pull Request na GitHubu

1) Pushni feature větev	git push -u origin feature/xyz
2) Na GitHubu: New Pull Request, feature → main	
3) Popiš změnu, projdi diff, okomentuj	
4) Merge / Squash and merge tlačítkem na GitHubu	
5) Doma stáhni výsledek a uklid' větev	git checkout main git pull git branch -d feature/xyz

Rebase / Cherry-pick / Stash

Přelož commity feature větve na aktuální main (jen na vlastní, ještě nesdílené větvi, ideálně před PR)	git checkout feature/xyz git rebase main
Konflikt při rebase → uprav soubor, pak	git add soubor git rebase --continue
Zrušit rozjetý rebase, vrátit stav před ním	git rebase --abort
Zkopírovat jeden konkrétní commit z jiné větve	git cherry-pick <hash>
Rychle odložit rozpracované změny (přepnutí větve)	git stash push -m "popis"
Vrátit odložené změny (smaže je ze seznamu)	git stash pop
Vypsat seznam / vrátit bez smazání	git stash list git stash apply

Reset vs. Revert (oprava chyby)

Chybný commit ještě NEpushnutý → vrátit HEAD, změny nechat needitované k opravě	git reset --mixed HEAD~1
...změny nechat i stagované (jen upravit zprávu)	git reset --soft HEAD~1
...zahodit změny úplně (POZOR, nevratné)	git reset --hard HEAD~1
Chybný commit UŽ pushnutý / sdílený → nový commit rušící jeho účinek	git revert <hash> git revert --no-edit <hash>
Editor při revert/rebase padá (vim)? Nastav natrvalo VS Code	git config --global core.editor "code --wait"

Tagy a Release

Vytvořit anotovaný tag na aktuálním commitu	git tag -a v1.0 -m "popis verze"
Poslat tag na GitHub (push sám tagy neposílá!)	git push origin v1.0
Zobrazit detail tagu	git show v1.0
GitHub: Releases → Draft a new release → vyber tag → Publish (přibalí zip/tar.gz, bez .git historie)	

Rychlé rozhodování - co použít

Situace	Řešení
Chci izolovat rozpracovanou funkci	nová větev (checkout -b)
Dvě větve upravily totéž místo	merge/rebase konflikt → ruční úprava
Chci čistou lineární historii před PR	rebase (jen na vlastní větvi)
Chci jen 1 commit odjinud, ne celou větev	cherry-pick
Musím rychle pryč, nechci commitovat	stash
Chyba v commitu, co jsem NEpushnul	reset --mixed / --soft
Chyba v commitu, co UŽ je na GitHubu	revert
Chci označit vydanou verzi	tag -a + push + GitHub Release

Tip: git status spouštěj před i po každém větším zásahu (reset, rebase, merge, revert) – hned uvidíš, co je rozpracované.

Klonování a práce s cizím repozitářem

Naklonování repozitáře z GitHubu - postup

1) Otevři ve VS Code rodičovskou složku, kam repo umístíš (File → Open Folder). Podslložku si nevytvářej, vznikne sama.	
2) Otevři terminál, over si, kde jsi	<code>pwd</code>
3) Na GitHubu: Code → HTTPS → zkopíruj URL repa	
4) Naklonuj – vytvoří se nová složka automaticky	<code>git clone <url></code>
5) Přepni se do naklonované složky	<code>cd nazev-repa</code>
6) Otevři tuto složku jako nové okno VS Code	<code>code .</code>
Kontrola: origin by měl ukazovat na URL, ze které jsi klonoval	<code>git remote -v</code>

Chci jen svou kopii (bez vazby na originál)

1) Uprav soubory a commitni normálně	<code>git add .</code> <code>git commit -m "..."</code>
2) Na GitHubu založ NOVÝ, zcela prázdný repozitář (bez README/.gitignore/licence!)	
3) Zjisti aktuální remote	<code>git remote -v</code>
4) Přesměruj origin na svůj nový repozitář	<code>git remote set-url origin <tvoje-nova-url></code>
5) Ověř přesměrování	<code>git remote -v</code>
6) Pošli celou historii na svůj repozitář	<code>git push -u origin main</code>

Chci zůstat propojený s originálem

Standardní cesta je Fork – GitHub ti na tvém účtu vytvoří kopii i s historií, kterou pak sám naklonuješ.

Detailní postup, sledování změn a řešení konfliktů → viz Fork na další straně.

Rychlé rozhodování - klonování

Situace	Postup
Jen chci lokální kopii svého vlastního repa	<code>git clone</code>
Cizí projekt, chci vlastní nezávislou kopii	<code>clone</code> → nový prázdný repo → <code>set-url origin</code> → <code>push</code>
Cizí projekt, chci přispívat / sledovat vývoj	Fork na GitHubu → <code>clone forku</code> → <code>upstream</code>

Fork - mechanika, sledování a řešení konfliktů

Jak funguje fork - mechanika

GitHub vytvoří na tvém účtu kopii celého repa (soubory + historie)

Fork a originál jsou od té chvíle NEZÁVISLÉ repozitáře - žádná živá synchronizace

GitHub si jen pamatuje vztah "fork z X" - umí ukázat rozdíl, nabídne tlačítko "Sync fork"

Jak se dozvíš o změnách v originálu

Automaticky NEjsi notifikován o každém commitu jen tím, že máš fork

Watch repozitář → notifikace o issues/PR/releases (podle nastavení)

Na stránce tvého forku vidíš hlášku "X commits behind" - při návštěvě

Nejjistější: sám pravidelně zkontroluj

```
git fetch upstream
git log upstream/main
--oneline
```

Chci zůstat propojený (fork + upstream)

1) Na GitHubu: tlačítko Fork na cizím repu	
2) Naklonuj SVŮJ fork (ne originál)	<code>git clone <url-tveho-forku></code>
3) Přidej originál jako druhý remote	<code>git remote add upstream <url-puvodniho-repa></code>
4) Stáhni novinky z originálu	<code>git fetch upstream</code>
5) Sluč je do své větve	<code>git merge upstream/main</code>
6) Pošli výsledek na svůj fork	<code>git push origin main</code>

Konfliktní scénář - oba upravili totéž

Vlastník originálu upraví divide() jinak, než ty na svém forku → po merge upstream/main nastane STEJNÝ typ konfliktu jako mezi dvěma branchemi (str. 2)

V souboru se objeví	<pre><<<<<< HEAD (tvoje verze) ===== (verze z upstream) >>>>>> upstream/main</pre>
Řešení: rozhodni finální podobu, smaž značky, ulož	<pre>git add soubor git commit</pre>
Pak pošli vyřešený stav na svůj fork	<code>git push origin main</code>

Tip: fork ti dá přehledný přístup k originálu (upstream), ale konflikt mezi vaší úpravou stejného místa souboru řešíš úplně stejně ručně jako u běžného merge mezi dvěma vlastními větvemi.